Unit I - Introduction to Operating System (09 Hours)

Basics of Operating Systems: Objectives & Functions, Evolution of OS, Types of Operating Systems, OS Service, System Calls: Introduction, Types of System Calls

OS structure: Layered Approach, Monolithic, Microkernel Operating Systems

Introduction to Linux OS: Components of Linux system, Basic Shell commands

Case studies:

1. Automating User and File Management in Linux using Shell Script
2. Demonstrating Different OS Structures (Monolithic, Layered, Microkernel OS)

ROHIT

Basics of Operating Systems:

Objectives of Operating System

- **Convenience:**
  The OS makes the computer easier to use by hiding hardware complexities from users.
  Example: When you open a file, the OS handles reading from the disk without you needing to know the storage hardware details.

- **Efficiency:**
  The OS optimizes use of hardware resources like CPU, memory, and I/O devices, improving overall system performance.
  Example: The OS allocates CPU time between running programs so no program hogs the processor.

- **Ability to Evolve:**
  The OS can be updated or extended with new features without disrupting existing user applications.
  Example: Adding support for a new printer model without changing how users code printing tasks.

Functions of Operating System

- **Process Management:**
  It manages processes by creating, scheduling, and terminating them. The OS allocates the CPU to multiple programs using time-sharing.
  Example: When you run multiple applications like a web browser and a word processor, the OS switches CPU time between them.

- **Memory Management:**
  The OS tracks and allocates space for programs in RAM and ensures no programs interfere with each other's memory.
  Example: When you open a game and a music player, the OS assigns them separate memory areas to prevent crashes.

- **File System Management:**
  It organizes and controls how files are stored, named, accessed, and secured on disks.
  Example: You save a document with a name and location; the OS manages storing it on the hard drive and retrieving it later.

ROHIT

- **Device Management:**
  The OS controls peripheral devices via drivers and allocates their use among programs.
  Example: When printing a file, the OS sends data to the printer driver, ensuring smooth print operation.

- **Security and Protection:**
  It enforces access controls and protects data from unauthorized access or malicious software.
  Example: Password login controls, file permissions, and antivirus integration are OS security features.

- **Error Detection and Handling:**
  The OS constantly checks for hardware or software errors and takes corrective actions or notifies the user.
  Example: If a disk read error occurs, the OS might retry or alert the user to a problem.

- **Resource Allocation:**
  The OS allocates system resources like CPU cycles, memory space, and I/O devices fairly among competing programs.
  Example: During heavy task loads, the OS prioritizes urgent tasks while keeping others running smoothly.

- **User Interface**
  Most OS provide a user interface (command line or graphical) that simplifies user interaction with the computer.
  Example: Windows Desktop, macOS Dock, or Linux terminal let users launch programs and manage files without complex commands.

- **Accounting:**
  It records system usage statistics to monitor resource usage for billing or performance tuning.
  Example: Tracking CPU time used by processes helps system administrators identify resource-intensive applications.

ROHIT

Evolution of OS :

The evolution of Operating Systems (OS) can be divided into distinct generations, reflecting the technological advancements and changing computing needs over time:

1. Early Computers (1940s-1950s): No OS

- Computers operated without an OS.
- Programs were manually loaded and run one at a time.
- The first OS was a simple batch processing system called GM-NAA I/O (1956) that automated job handling.

2. Batch Processing Systems (1955-1965)

- Introduced the concept of batch processing: jobs were grouped and run sequentially without user interaction.
- Improved efficiency by automating job sequencing.
- Examples: IBM's early OS like IBSYS.

3. Multiprogramming and Time-Sharing Systems (1960s-1970s)

- Multiprogramming allowed multiple programs to reside in memory, improving CPU utilization.
- Time-sharing systems enabled many users to interact with a computer simultaneously.
- Examples: CTSS (Compatible Time Sharing System, 1961), Multics (1969).

4. Personal Computer OS and Unix (1970s-1980s)

- Unix, developed in 1971, introduced simplicity, portability, and multitasking.
- Emergence of personal computers led to simpler OSs like CP/M (1974) and MS-DOS (1981).
- Graphical User Interfaces (GUI) began developing with Apple Macintosh (1984) and Microsoft Windows (1985).

5. Networking and GUI Expansion (1980s-1990s)

- OSs incorporated networking capabilities and refined GUIs.
- Windows and Mac OS became widely adopted.
- Networking protocols like TCP/IP integrated.

6. Open Source and Modern OS (1990s-Present)

ROHIT

- Linux (1991) popularized open-source development.
- Mobile OSs emerged: iOS (2007), Android (2008).
- Cloud computing and virtualization became integrated OS features.
- Modern OS continue evolving with enhanced security, performance, and support for diverse hardware.

Period	Key Developments	Examples
1940s-1950s	No OS, manual programming	GM-NAA I/O (1956)
1955-1965	Batch Processing	IBSYS
1960s-1970s	Multiprogramming, Time-sharing	CTSS, Multics
1970s-1980s	Unix, Personal Computers	Unix, CP/M, MS-DOS
1980s-1990s	GUI, Networking	Windows, Mac OS
1990s-Present	Open Source, Mobile, Cloud	Linux, Android, iOS

Types of Operating Systems:

Operating Systems (OS) can be classified into several main types based on how they manage users, tasks, hardware, and interactions. Here is an overview of the most important types along with examples, useful for exam preparation:

1. Batch Operating System

- Executes batches of similar jobs without user interaction during processing.
- Jobs are collected, queued, and run in sequence.
- Used in early mainframe computers for payroll, data entry, etc.
- Example: IBM's early batch systems.

2. Multi-Programming Operating System

- Allows multiple programs to reside in memory at the same time.
- CPU switches among programs to optimize resource use.
- Increases CPU utilization by avoiding idle time.
- Example: Early OS supporting multiprogramming.

3. Time-Sharing/Multitasking Operating System

- Enables many users or programs to share system resources concurrently.
- Uses CPU time slicing to provide responsiveness.
- Supports interactive use with graphical or command line interfaces.
- Examples: Windows, macOS, Linux.

4. Distributed Operating System

- Manages a group of distinct computers and makes them appear as a single system.
- Coordinates resource sharing and communication over a network.
- Example: Amoeba, Google’s Android (to some extent).

5. Network Operating System

- Provides features to support workstations, printers, and other devices on a network.
- Manages network traffic and user permissions.
- Examples: Novell NetWare, Windows Server.

ROHIT

6. Real-Time Operating System (RTOS)

- Provides guaranteed response times for critical tasks.
- Used in embedded systems, robotics, medical devices, and avionics.
- Example: VxWorks, QNX.

7. Mobile Operating System

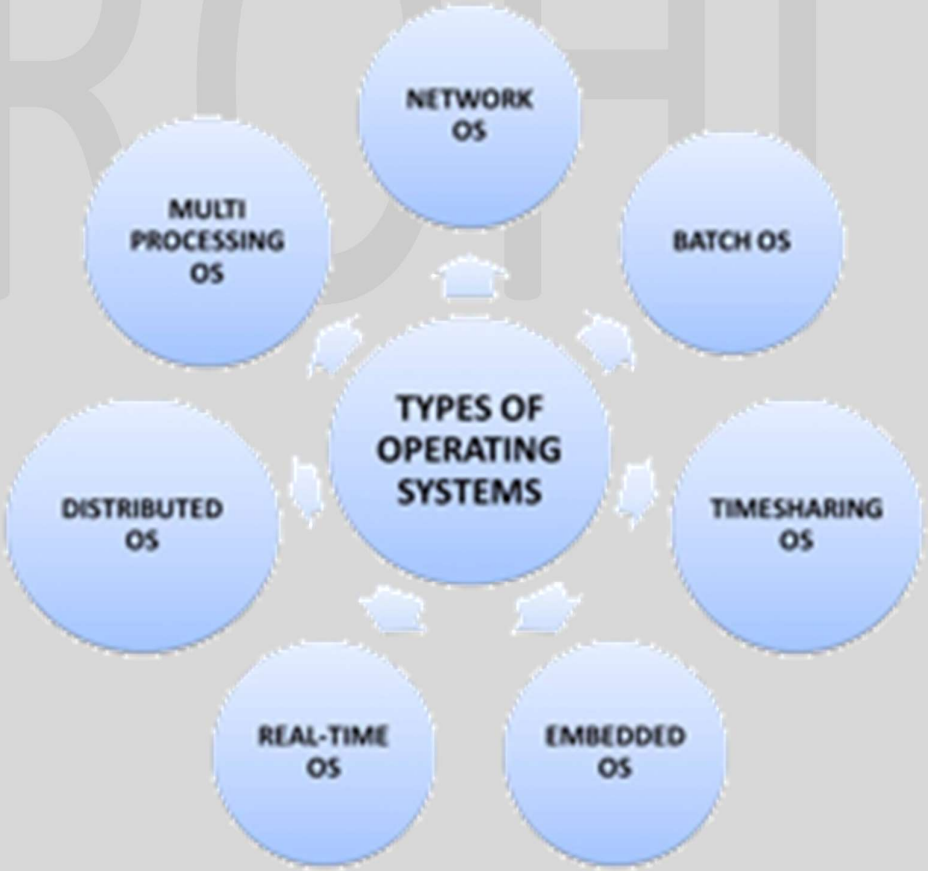
- Designed specifically for mobile devices with touch interfaces and battery efficiency.
- Examples: Android, iOS.

8. Embedded Operating System

- Specialized OS designed for embedded systems with limited resources.
- Focus on stability, minimal resource usage, and real-time capabilities.
- Examples: Embedded Linux, FreeRTOS.

Type	Key Feature	User Interaction	Examples	Common Uses
Batch OS	Batch job processing	None	IBM Batch Systems	Data processing, payroll
Multiprogramming OS	Concurrent program execution	Limited	Early multiprogramming OS	High CPU utilization
Time-sharing OS	CPU time shared interactively	Multiple interactive users	Windows, Linux, macOS	General purpose computing
Distributed OS	Networked multiple systems	Distributed management	Amoeba	Cluster computing, cloud
Network OS	Network resource management	Networked users	Windows Server, Novell	Enterprise networks

Type	Key Feature	User Interaction	Examples	Common Uses
Real-Time OS (RTOS)	Guaranteed timing	Embedded interaction	VxWorks, QNX	Robotics, medical systems
Mobile OS	Mobile device optimization	Touch interface	Android, iOS	Smartphones, tablets
Embedded OS	Minimal resource OS	Specialized devices	Embedded Linux, FreeRTOS	IoT, automotive, appliances



OS Service:

Operating System Services are the essential functions and features provided by the OS to make the computer system usable and efficient for users and applications. Here are the main OS services with explanations and examples:

1. Program Execution

- The OS manages the execution of programs by loading them into memory and scheduling them for CPU time.
- Example: When you launch a browser, the OS allocates resources and CPU time so it runs smoothly alongside other applications.

2. Input/Output Operations

- It facilitates communication between hardware devices (keyboard, mouse, disk, printer) and software, handling I/O requests and transferring data.
- Example: Printing a document involves the OS controlling the printer hardware and sending print jobs.

3. File System Manipulation

- The OS manages file creation, deletion, reading, writing, and organization on storage devices.
- Example: Saving a photo on your hard drive and retrieving it later are handled by the file management service.

4. Communication

- Enables processes and users to exchange information through mechanisms like message passing or shared memory.
- Example: Two programs collaborating in a word processor and spreadsheet exchange data using OS communication services.

5. Error Detection and Handling

- Detects errors in hardware or software and takes corrective action or notifies users.
- Example: If a disk read error occurs, the OS will try to recover data or alert the user.

6. Resource Allocation

- Allocates and manages hardware resources such as CPU cycles, memory, and I/O devices among multiple competing processes.

ROHIT

- Example: When multiple apps run at once, the OS decides who gets CPU time and how much memory to allocate.

7. Security and Protection

- Protects data and system resources from unauthorized access and malware through authentication, permissions, and encryption.
- Example: Password protection at login and file access controls.

8. User Interface

- Provides an interface for user interaction, either through graphical user interfaces (GUI) or command-line interfaces (CLI).
- Example: Desktop environment in Windows or terminal in Linux.

9. Job Accounting and System Performance Monitoring

- Tracks resource usage by users/processes for auditing, billing, or optimizing system performance.
- Example: Logging CPU time used by programs to identify resource-heavy applications.

ROHIT

System Calls: Introduction, Types of System Calls:

System Calls in an Operating System are special functions that act as an interface between user programs (applications) and the operating system kernel. They allow programs to request services from the OS that cannot be directly accessed by user applications due to privilege and security reasons.

Introduction to System Calls:

- System calls provide a controlled mechanism for programs to interact with hardware and system resources.
- They enable operations like process control, file management, device handling, communication, and security.
- When a program makes a system call, it interrupts the program flow and switches the CPU to kernel mode to execute the requested service securely.
- Examples of system call usage include creating or terminating processes, reading/writing files, or communicating over a network.

Types of System Calls:

1. Process Control System Calls:
    - Operations related to process creation, termination, and management.
    - Examples: fork() (create a process), exit() (terminate process), wait() (wait for process), exec() (execute new program).
2. File Management System Calls:
    - Manage file operations like open, read, write, close, delete.
    - Examples: open(), read(), write(), close(), unlink().
3. Device Management System Calls:
    - Control and interact with hardware devices.
    - Examples: ioctl() (device-specific operations), read(), write(), open() device files.
4. Information Maintenance System Calls:
    - Retrieve and update system information such as time and user info.
    - Examples: gettimeofday(), settimeofday(), getpid().
5. Communication System Calls:

ROHIT

- Facilitate communication between processes, either on the same or different machines.
- Examples: socket(), bind(), connect(), send(), recv().

6. Security and Access Control System Calls:
    - Manage permissions and control access to system resources.
    - Examples: chmod() (change permissions), chown() (change ownership).

System calls vary in names and exact behavior depending on the operating system, but their purposes are generally consistent across platforms like Unix/Linux and Windows.

ROHIT

OS structure:

Layered Approach, Monolithic, Microkernel Operating Systems:

Operating system architecture defines how the OS components are organized and interact. The key structures are the Layered Approach, Monolithic, and Microkernel operating systems:

Layered Approach

- The OS is divided into layers, each built on top of the one below it.
- Lower layers manage hardware, while higher layers provide user services and interfaces.
- Each layer only interacts directly with the layer below and above it, which simplifies design and debugging.
- Example: MULTICS uses a layered structure.
- Advantages: Easier to manage, maintain, and debug due to clear boundaries.
- Disadvantages: Layer communication overhead can reduce performance.

Monolithic Operating System

- All OS services like process management, memory, file system, and device drivers run in a single kernel space.
- Components communicate through direct function calls without strict separation.
- Fast and efficient because everything operates in kernel mode.
- Examples: Early UNIX, Linux, MS-DOS.
- Advantages: High performance due to no context switching between modules.
- Disadvantages: Difficult to maintain, debug, and less secure since a bug in one service can crash the entire system.

Microkernel Operating System

- The kernel contains only essential core functions: low-level address space management, thread management, and inter-process communication.
- Other OS services such as device drivers, file systems, and network protocols run in user space as separate processes.

ROHIT

- This modularity leads to better stability and security because failures in user-space services do not crash the whole OS.
- Examples: MINIX, QNX, macOS (hybrid microkernel), Windows NT (hybrid).
- Advantages: Better fault isolation, easier to extend and maintain.
- Disadvantages: Potentially slower due to increased context switching and communication between kernel and user services.

ROHIT

Introduction to Linux OS:

Components of Linux System

1. **Kernel:**
   The core of Linux, it interacts directly with hardware like CPU, memory, and I/O devices. The kernel manages process scheduling, memory allocation, device drivers, file systems, and network communication. It provides system calls that link software and hardware.

2. **Shell:**
   The command-line interface between the user and the kernel. It interprets user commands and passes them to the kernel for execution. Common shells include Bash (Bourne Again Shell), C Shell, and Korn Shell.

3. **System Libraries:**
   Set of functions and routines that programs can use to communicate with the kernel, without needing to perform system calls directly. For example, GNU C Library (glibc) provides basic C functions.

4. **System Utilities:**
   Essential programs for system management and maintenance like file manipulation commands (ls, cp, rm), process management (ps, kill), and administrative tools (useradd, systemctl).

5. **Hardware:**
   Physical devices like CPU, RAM, hard drives, keyboard, and network cards, which the kernel manages.

Basic Shell Commands in Linux

- ls : Lists files and directories in the current directory.
- cd [directory]: Changes the current directory.
- pwd: Prints the current working directory.
- cp [source] [destination]: Copies files or directories.
- mv [source] [destination]: Moves or renames files or directories.
- rm [file]: Removes files or directories.
- mkdir [directory]: Creates a new directory.
- rmdir [directory]: Removes an empty directory.

ROHIT

- cat [file]: Displays the content of a file.
- echo [text]: Displays a line of text.
- ps: Displays running processes.
- kill [pid]: Terminates a process by its process ID.
- chmod [options] [file]: Changes file permissions.
- man [command]: Shows the manual page of a command.

ROHIT

Explain about the dual mode operation in OS with a neat block diagram.

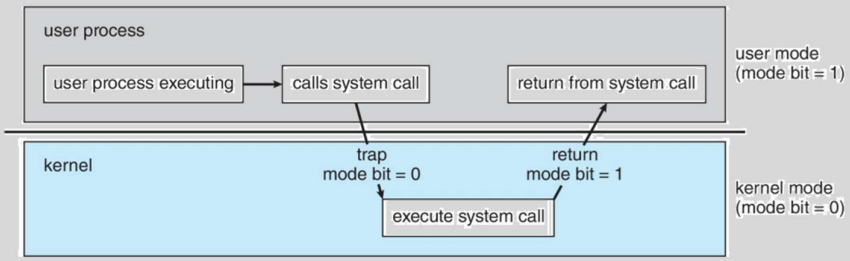
Dual mode operation in an operating system is a fundamental concept that enhances security and stability by dividing CPU execution into two distinct modes: **User Mode** and **Kernel Mode**.

Explanation of Dual Mode Operation

- **User Mode:**
  In this mode, user applications run with restricted privileges. They cannot directly access hardware or critical system resources. If a user process tries to perform a privileged operation (like accessing hardware), it must make a system call to request the OS to do it. This protects the system from accidental or intentional damage by user-level programs.
- **Kernel Mode:**
  The operating system runs in this mode with full control over the hardware, memory, and all critical resources. It can execute privileged instructions and manage system resources. When a user request is made (via a system call or interrupt), the CPU switches from user mode to kernel mode to safely carry out the operation, then switches back to user mode.
- **Mode Bit:**
  The CPU uses a special mode bit to distinguish between these two modes (0 for kernel mode, 1 for user mode). Switching this bit ensures only trusted OS code can run critical operations.

ROHIT

Transition from User to Kernel Mode



Define system calls. Explain. *fork( )* system call ?

Definition of System Calls:
System calls are special functions that provide a controlled interface between a user program (running in user mode) and the operating system kernel (running in kernel mode). They allow programs to request OS services for tasks like file operations, process control, device management, and inter-process communication, which cannot be performed directly due to system security and privilege boundaries. When a system call is made, the CPU switches from user mode to kernel mode to execute the request, then returns control to the user program.

fork() System Call Explanation:
The fork() system call in Unix/Linux creates a new process by duplicating the calling (parent) process. After the fork() call, two processes exist:

- **Parent Process:** The original process that invoked fork()
- **Child Process:** The newly created process, which is an exact copy of the parent (except for unique identifiers like PID)

Both processes continue executing independently from the point where fork() was called. The child inherits code, data, and environment from the parent process. The key distinction is in the return value:

- In the parent process, fork() returns the child's process ID (PID)
- In the child process, fork() returns 0

ROHIT